

Übung 6: Projektweiterentwicklung und Aspect Oriented Programming (AOP)

Team 6 (Thema E-Vote) - Abgabe:

Link: https://github.com/jacque-schu/MSE_eVote

Gemeinsam als Team haben wir uns über Discord zum Thema „Projektweiterentwicklung und Aspect Oriented Programming (AOP)“ ausgetauscht. Hier tragen wir gemeinsam unsere Antworten zusammen.

Aufgabe 1: Backend-Weiterentwicklung:

Siehe GitHub Repository

Aufgabe 2: Frontend-Weiterentwicklung:

Siehe GitHub Repository

Aufgabe 3: Aspect Oriented Programming (AOP):

In unserem Projekt zur digitalen Abstimmungsplattform bildet das Domain-Driven-Design-Modell (DDD) die Grundlage für die Identifikation von Querschnittsaufgaben und potenziellen AOP-Anwendungsfällen. Das Modell unterscheidet klar zwischen den Bounded Contexts „Bürgerverwaltung“ und „Abstimmungsmanagement“ sowie deren Aggregaten wie Bürger, Abstimmung und Ergebnis. Dadurch werden fachliche Verantwortlichkeiten transparent und wiederkehrende technische oder fachliche Aufgaben fallen deutlich auf.

Auf Basis dieses Modells lassen sich mehrere zentrale Querschnittsbelange erkennen: Sicherheits- und Rechteprüfungen (Nutzer muss registriert und wahlberechtigt sein), technische Authentifizierung, Logging wichtiger Aktionen (Registrierung, Stimmabgabe, Ergebnisabfrage), Fehlerbehandlung, Auditing sowie gegebenenfalls Transaktionssteuerung rund um schreibende Vorgänge. Diese Aufgaben betreffen viele Use Cases gleichzeitig, gehören aber nicht zur eigentlichen Fachlogik der Abstimmung oder der Bürgerverwaltung.

Ebenfalls sichtbar werden wiederholende Funktionalitäten, etwa die Prüfung, ob ein Bürger registriert und wahlberechtigt ist, bevor er eine Stimme abgeben oder ein

Ergebnis einsehen darf. Diese Logik tritt in mehreren Application Services auf (z.B. RegistrierungsService, AbstimmungsService, AbstimmungsübersichtService) und eignet sich daher ideal zur Zentralisierung.

Aus dieser Analyse ergeben sich konkrete AOP-Anwendungsfälle. Besonders im Fokus steht der Security-Aspekt zur Prüfung der Nutzerrechte. Dieser Aspekt wird vor allen sicherheitsrelevanten Methoden ausgeführt und stellt sicher, dass nur registrierte und berechtigte Bürger Aktionen wie „stimmeAbgeben()“ oder das Abrufen der Abstimmungsergebnisse ausführen können. Anstatt die Rechteprüfung in jedem Service einzeln zu implementieren, wird sie als wiederverwendbarer Aspekt definiert und automatisch in den Aufruffluss der Application Services „eingewoben“. Damit bleibt das Domain-Modell frei von technischer Sicherheitslogik, die Querschnittsaufgabe „Nutzerrechte prüfen“ ist an einer Stelle zentral wartbar, und das Gesamtsystem bleibt klar strukturiert und erweiterbar

Durch die gesamte AOP-Integration wurde eine modulare und wiederverwendbare Authentifizierungsarchitektur geschaffen, die Authentifizierungsprüfungen zentral in einem Aspect kapselt und automatisiert an beliebigen Service-Methoden anwenden lässt, ohne deren Geschäftslogik zu beeinträchtigen. Diese Implementierung kann zukünftig einfach auf weitere Service-Methoden oder Business Contexts erweitert werden, indem der @auth_check-Dekorator hinzugefügt wird, ohne dass bestehender Code modifiziert werden muss. Das System schützt die Anwendung vor unbefugten Zugriffen, indem es gewährleistet, dass nur Anfragen mit gültigen Authentifizierungstokens die geschützten Service-Methoden erreichen, und wirft automatisch HTTP-401-Fehler bei fehlenden oder ungültigen Tokens ab.

Aufgabe 5: LLM-Einsatz für AOP:

In unserem DDD-basierten Projekt zur digitalen Abstimmungsplattform haben wir Large Language Models wie GPT-5 verwendet, um AOP-Anwendungen zu identifizieren.

LLMs unterstützen dabei die dynamische Identifikation und Generierung von Aspekten, indem sie Code analysieren und Vorschläge in natürlicher Sprache liefern. Dies verbessert die Trennung von Fachlogik (z.B. Abstimmungsaggregate) und Querschnittsbelangen (z.B. Security in Application Services).

(a) Dokumentation des LLM-Einsatzes:

- **Identifikation von Cross-cutting Concerns:**

Mit Hilfe des LLMs haben wir den Quellcode und die DDD-Struktur analysieren lassen. Das LLM konnte Mustermatching betreiben, um wiederkehrende Aufgaben (wie die Prüfung der Nutzerrechte, Logging und Validierung) als cross-cutting zu erkennen.

Beispielsweise wurde sichtbar, dass in mehreren Application Services geprüft wird, ob ein Bürger registriert ist. Das LLM empfahl, diese Prüfung in einem Security-Aspekt zu bündeln.

- **Optionale Entwicklung von Aspekten:**

Für Python wurde das LLM eingesetzt, um gezielt AOP-Patterns mithilfe von Decorators oder Bibliotheken wie aspectlib, aspects.py oder AOPython zu entwickeln. Das Modell generierte passende Decorator-Funktionen, etwa einen @check_user_rights-Decorator, der vor kritischen Methoden automatisch die Berechtigungen prüft.

LLM-basierte Analysen halfen auch, die optimale „Einwebung“ (Weaving) von Aspekten an strategischen Join Points (z.B. Methoden wie stimme_abgeben() oder zeige_ergebnisse()) vorzuschlagen.

(b) Reflexion: Nutzen, Herausforderungen, Learnings

- **Nützlichkeit:**

Das LLM beschleunigte die Identifikation und Implementierung von Aspekten. Routineaufgaben wie die Wiederverwendung von Decorators oder das Erstellen von AOP-konformen Validierungslogiken können dadurch effizient automatisiert werden.

Die Konsistenz im Code wird dadurch verbessert, und die Trennung von Fach- und Techniklogik wird besser umgesetzt.

- **Herausforderung:**

Viele Lösungsvorschläge waren zunächst zu allgemein oder zu „lehrbuchhaft“.

Gerade bei unserem konkreten DDD-Modell und der Python-AOP-Umsetzung (Decorators, konkrete Methoden wie stimme_abgeben/zeige_ergebnisse) mussten wir die Vorschläge durch gezieltes Nachfragen und Verfeinern der Prompts schrittweise konkretisieren.

- **Lösungen & Learnings:**

Prompt-Engineering, eigene Projekt-Templates und gezielte Rückversicherung durch manuelle Code-Reviews sichern die Qualität.

Unser Learning: LLMs sind starke Werkzeuge zur Beschleunigung von Analyse und Prototyping, ersetzen aber noch keine tiefen Python-Kenntnisse und eine kritische Prüfung im Kontext des eigenen DDD-Modells.